Practical No.01

Aim: Write a program to implement logical gates AND, OR and NOT with McCulloch-Pitts

Code:

```
def mp_neuron(inputs, weights, threshold):
    y_in = sum(xi * wi for xi, wi in zip(inputs, weights))
    if y_in >= threshold:
        output = 1
    else:
        output = 0
    return output

def AND_gate(x1,x2):
    weights = [1, 1]
    threshold = 2
    inputs = [x1, x2]
    return mp_neuron(inputs, weights, threshold)

def OR_gate(x1,x2):
    weights = [1, 1]
    threshold = 1
    inputs = [x1, x2]
    return mp_neuron(inputs, weights, threshold)

def NOT_gate(x1):
    weights = [-1]
    threshold = 2
    inputs = [x1]
    return mp_neuron(inputs, weights, threshold)

def test_gates():
    print("AND GATE")
    for x1 in [0, 1]:
        for x2 in [0, 1]:
            print(f"{x1},{x2}: {AND_gate(x1, x2)}")

    print("\nOR GATE")
    for x1 in [0, 1]:
        for x2 in [0, 1]:
            print(f"{x1},{x2}:{OR_gate(x1, x2)}")

    print("\nNOT GATE")
    for x1 in [0, 1]:
        for x2 in [0, 1]:
            print(f"{x1}:{NOT_gate(x1)}")

test_gates()
```

Output:

```
>>>
========== RESTART: C:/Users/SIES COLLEGE/scpract
AND GATE
0,0: 0
0,1: 0
1,0: 0
1,1: 1

OR GATE
0,0:0
0,1:1
1,0:1
1,1:1

NOT GATE
0:0
0:0
1:0
1:0
>>>
```

Practical No 02

Aim: Write a program to implement Hebb‟s rule.

Code:

```
inputs = [
    [-1, -1, -1],
    [-1, 1, 1],
    [1, -1, 1],
    [1, 1, 1]
]

targets = [-1, -1, -1, 1]

weights = [0, 0, 0]

print("Training using Hebbian Learning Rule:\n")

for i in range(4):
    x = inputs[i]
    t = targets[i]
    print(f"Input {i+1}: {x}, Target: {t}")

    for j in range(3):
        weights[j] += x[j] * t
    print(f"Updated Weights: {weights}\n")

print("Final Weights:", weights)

print("\nTesting the network with final weights:")

for x in inputs:
    y = sum(i * w for i, w in zip(x, weights))
    print(f"Input: {x[:-1]}, Bias: {x[2]}, Output: {y}")
```

Output:

```
Training using Hebbian Learning Rule:

Input 1: [-1, -1, -1], Target: -1
Updated Weights: [1, 1, 1]

Input 2: [-1, 1, 1], Target: -1
Updated Weights: [2, 0, 0]

Input 3: [1, -1, 1], Target: -1
Updated Weights: [1, 1, -1]

Input 4: [1, 1, 1], Target: 1
Updated Weights: [2, 2, 0]

Final Weights: [2, 2, 0]

Testing the network with final weights:
Input: [-1, -1], Bias: -1, Output: -4
Input: [-1, 1], Bias: 1, Output: 0
Input: [1, -1], Bias: 1, Output: 0
Input: [1, 1], Bias: 1, Output: 4
```

Practical No 03

Aim: Implement Kohonen Self organizing map.

Code:

```
import numpy as np
data = np.array([
    [0, 0, 1, 1],
    [1, 0, 0, 0],
    [0, 1, 1, 0],
    [0, 0, 0, 1]
])

weights = np.array([
    [0.2, 0.9],
    [0.4, 0.7],
    [0.6, 0.5],
    [0.8, 0.3]
])

alpha = 0.5

print("KSOFM Training:")

for idx, x in enumerate(data, start=1):
    dists = np.sum((weights - x[:, np.newaxis]) ** 2, axis=0)
    winner = np.argmin(dists)

    print(f"\nInput x{idx} = {x}, Distances = {np.round(dists,4)}, Winner Neuron = {winner+1}")

    for i in range(len(x)):
        weights[i, winner] += alpha * (x[i] - weights[i, winner])
    formatted_weights = ["{:.4f}".format(weights[i, winner]) for i in range(len(x))]
    print(f"Updated weights for Neuron {winner+1}: {formatted_weights}")

print("\nFinal Weights After One Pass:")

for i in range(weights.shape[0]):
    row = ["{:.4f}".format(val) for val in weights[i]]
    print(f"[{row[0]}, {row[1]}]")
```

Output:

```
= RESTART: C:/Users/admin/AppData/Local/Programs/Python/Python312/practe.py
KSOFM Training:

Input x1 = [0 0 1 1], Distances = [0.4  2.04], Winner Neuron = 1
Updated weights for Neuron 1: ['0.1000', '0.2000', '0.8000', '0.9000']

Input x2 = [1 0 0 0], Distances = [2.3  0.84], Winner Neuron = 2
Updated weights for Neuron 2: ['0.9500', '0.3500', '0.2500', '0.1500']

Input x3 = [0 1 1 0], Distances = [1.5  1.91], Winner Neuron = 1
Updated weights for Neuron 1: ['0.0500', '0.6000', '0.9000', '0.4500']

Input x4 = [0 0 0 1], Distances = [1.475 1.81 ], Winner Neuron = 1
Updated weights for Neuron 1: ['0.0250', '0.3000', '0.4500', '0.7250']

Final Weights After One Pass:
[0.0250, 0.9500]
[0.3000, 0.3500]
[0.4500, 0.2500]
[0.7250, 0.1500]
```

Practical No 04

Aim: Solve the Hamming network given the exemplar vectors.

Code

```
import numpy as np

e1 = [1, -1, -1, -1]
e2 = [-1, -1, -1, 1]

w = 0.5 * np.array([e1, e2]).T

print("Weight Matrix (w):\n", w)

bias = np.array([2, 2])

x = np.array([
    [-1, -1, 1, -1],
    [-1, -1, 1, 1],
    [-1, -1, -1, 1],
    [1, 1, -1, -1]
])

def hamming_net_output(x, w, bias):
    return bias + np.dot(x, w)

def maxnet(y, epsilon=0.1, max_iter=10):

    y = np.copy(y)

    for _ in range(max_iter):

        new_y = np.copy(y)

        for i in range(len(y)):
            inhibition = epsilon * sum(y[j] for j in range(len(y)) if j != i)
            new_y[i] = max(0, y[i] - inhibition)

        if np.allclose(new_y, y):
            break

        y = new_y

    return y

for i, x_i in enumerate(x):

    print(f"\nInput x{i+1} = {x_i}")
```

```
y_raw = hamming_net_output(x_i, w, bias)
print(f"Initial Output (y_in) = {y_raw}")

y_final = maxnet(y_raw)
print(f"After Maxnet = {y_final}")

winner = np.where(y_final == np.max(y_final))[0]

if len(winner) == 1:
    print(f"Assigned to cluster: e{winner[0] + 1}")
else:
    print(f"Ambiguous match: Possibly e{winner[0] + 1} and e{winner[1] + 1}")
```

Output:

```
Weight Matrix (w):
 [[ 0.5 -0.5]
 [-0.5 -0.5]
 [-0.5 -0.5]
 [-0.5  0.5]]

Input x1 = [-1 -1  1 -1]
Initial Output (y_in) = [2. 2.]
After Maxnet = [0.69735688 0.69735688]
Ambiguous match: Possibly e1 and e2

Input x2 = [-1 -1  1  1]
Initial Output (y_in) = [1. 3.]
After Maxnet = [0.     2.7763]
Assigned to cluster: e2

Input x3 = [-1 -1 -1  1]
Initial Output (y_in) = [2. 4.]
After Maxnet = [0.       3.365884]
Assigned to cluster: e2

Input x4 = [ 1  1 -1 -1]
Initial Output (y_in) = [3. 1.]
After Maxnet = [2.7763 0.    ]
Assigned to cluster: e1
```

Practical No. 5

Aim: Write a program for implementing BAM networks.

Code:

```
import numpy as np

def bipolar_sign(x):
   return np.where(x >= 0, 1, -1)

X = np.array([
   [1, 1, 1, 1, -1, -1, 1, 1, 1, -1, 1, -1, 1, 1, 1],
   [1, 1, 1, 1, -1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1]
])

Y = np.array([
   [-1, 1],
   [1, 1]
])
W = X.T @ Y

print("Weight Matrix W:\n", W)

def recall_bam(x_input, W, max_iters=10):
   x = x_input.copy()
   for _ in range(max_iters):
      y = bipolar_sign(x @ W)
      x_new = bipolar_sign(y @ W.T)
      if np.array_equal(x_new, x):
         break
      x = x_new

   return x, y
for idx, x in enumerate(X):

   x_out, y_out = recall_bam(x, W)

   print(f"\nInput Pattern {chr(69 + idx)}: {x}")
   print(f"Recalled Input X: {x_out}")
   print(f"Recalled Output Y: {y_out}")
```

Output:

```
Weight Matrix W:
 [[ 0  2]
 [ 0  2]
 [ 0  2]
 [ 0  2]
 [ 0 -2]
 [ 0 -2]
 [ 0  2]
 [-2  0]
 [ 0  2]
 [ 0 -2]
 [ 0  2]
 [ 0 -2]
 [ 0  2]
 [-2  0]
 [ 0  2]]

Input Pattern E: [ 1  1  1  1 -1 -1  1  1  1 -1  1 -1  1  1  1]
Recalled Input X: [ 1  1  1  1 -1 -1  1  1  1 -1  1 -1  1  1  1]
Recalled Output Y: [-1  1]

Input Pattern F: [ 1  1  1  1 -1 -1  1 -1  1 -1  1 -1  1 -1  1]
Recalled Input X: [ 1  1  1  1 -1 -1  1 -1  1 -1  1 -1  1 -1  1]
Recalled Output Y: [1 1]
```

Practical No 06

Aim: Implement a program to find the winning neuron using MaxNet.

Code:

```
import numpy as np
def MaxNet():
    activations = np.array([0.3,0.5,0.7,0.9])
    epsilon = 0.2
    m = len(activations)
    iteration = 0
    print("Initial Activations:",activations)

    while np.count_nonzero(activations) > 1:
        new_activations = np.zeros_like(activations)
        for j in range(m):
            inhibition = epsilon * sum(activations[i] for i in range(m) if i != j)
            new_activations[j] = activations[j] - inhibition
            new_activations[j] = max(0,new_activations[j])
        activations = new_activations
        iteration += 1
        print(f"After iteration {iteration}: {activations}")

    winner_index = np.argmax(activations)
    print(f"\nWinning Neuron: Neuron {winner_index + 1} with final activations {activations[winner_index]}")

MaxNet()
```

Output:

```
=== RESTART: C:/Users/admin/AppData/Local/Programs/Python/Python312
/pract6.py ==
Initial Activations: [0.3 0.5 0.7 0.9]
After iteration 1: [0.   0.12 0.36 0.6 ]
After iteration 2: [0.    0.    0.216 0.504]
After iteration 3: [0.     0.     0.1152 0.4608]
After iteration 4: [0.      0.      0.02304 0.43776]
After iteration 5: [0.       0.       0.       0.433152]

Winning Neuron: Neuron 4 with final activations 0.4331520000000001
```

Practical No.07

Aim: Implement De-Morgan‟s Law

Code:

```
def fuzzy_union(A, B):
    return {
        x: max(A.get(x, 0), B.get(x, 0))
        for x in sorted(set(A) | set(B))
    }

def fuzzy_intersection(A, B):
    return {
        x: min(A.get(x, 0), B.get(x, 0))
        for x in sorted(set(A) & set(B))
    }

def fuzzy_complement(A):
    return {
        x: 1 - A[x]
        for x in A
    }

A = {
    'x1': 0.2,
    'x2': 0.7,
    'x3': 1.0
}

B = {
    'x1': 0.5,
    'x2': 0.4,
    'x3': 0.6
}

lhs1 = fuzzy_complement(fuzzy_union(A, B))
rhs1 = fuzzy_intersection(
    fuzzy_complement(A),
    fuzzy_complement(B)
)

lhs2 = fuzzy_complement(fuzzy_intersection(A, B))
rhs2 = fuzzy_union(
    fuzzy_complement(A),
    fuzzy_complement(B)
)

print("Fuzzy Set A:", A)
print("Fuzzy Set B:", B)
```

```
print("\nFirst De Morgan's Law:")
print("(A Union B)' =", lhs1)
print("A' Intersection B' =", rhs1)
print("Verified:", lhs1 == rhs1)

print("\nSecond De Morgan's Law:")
print("(A Intersection B)' =", lhs2)
print("A' Union B' =", rhs2)
print("Verified:", lhs2 == rhs2)

print("\nDe Morgan's Law verification:")
print("(A Union B)' == (A' Intersection B'):", lhs1 == rhs1)
print("(A Intersection B)' == (A' Union B'):", lhs2 == rhs2)
print("Both laws verified:", lhs1 == rhs1 and lhs2 == rhs2)
```

Output:

```
== RESTART: C:/Users/91908/AppData/Local/Programs/Python/Python311/pract
Fuzzy Set A: {'x1': 0.2, 'x2': 0.7, 'x3': 1.0}
Fuzzy Set B: {'x1': 0.5, 'x2': 0.4, 'x3': 0.6}

First De Morgan's Law:
(A Union B)' = {'x1': 0.5, 'x2': 0.30000000000000004, 'x3': 0.0}
A' Intersection B' = {'x1': 0.5, 'x2': 0.30000000000000004, 'x3': 0.0}
Verified: True

Second De Morgan's Law:
(A Intersection B)' = {'x1': 0.8, 'x2': 0.6, 'x3': 0.4}
A' Union B' = {'x1': 0.8, 'x2': 0.6, 'x3': 0.4}
Verified: True

De Morgan's Law verification:
(A Union B)' == (A' Intersection B'): True
(A Intersection B)' == (A' Union B'): True
Both laws verified: True
```

Practical no.08

Aim: Implement Union, Intersection, Complement and Difference operations on fuzzy sets.

Code:

```
def fuzzy_union(A, B):
    return {x: max(A.get(x, 0), B.get(x, 0)) for x in set(A) | set(B)}

def fuzzy_intersection(A, B):
    return {x: min(A.get(x, 0), B.get(x, 0)) for x in set(A) | set(B)}

def fuzzy_complement(A):
    return {x: 1 - A[x] for x in A}

def fuzzy_difference(A, B):
    B_comp = fuzzy_complement(B)
    return fuzzy_intersection(A, B_comp)

if __name__== "__main__":
    A = {"x1": 1.0, "x2": 0.7, "x3": 1.0}
    B = {"x1": 0.5, "x2": 0.4, "x3": 0.6}

    print("Fuzzy Set A :", A)
    print("Fuzzy Set B :", B)
    print("\nUnion (A ∪ B) :", fuzzy_union(A, B))
    print("Intersection (A ∩ B) :", fuzzy_intersection(A, B))
    print("Complement (A') :", fuzzy_complement(A))
    print("Difference (A - B) :", fuzzy_difference(A, B))
```

Output:

```
======================== RESTART: C:/Users/Admin/pract8.py ==========
Fuzzy Set A : {'x1': 1.0, 'x2': 0.7, 'x3': 1.0}
Fuzzy Set B : {'x1': 0.5, 'x2': 0.4, 'x3': 0.6}

Union (A ∪ B) : {'x2': 0.7, 'x1': 1.0, 'x3': 1.0}
Intersection (A ∩ B) : {'x2': 0.4, 'x1': 0.5, 'x3': 0.6}
Complement (A') : {'x1': 0.0, 'x2': 0.30000000000000004, 'x3': 0.0}
Difference (A - B) : {'x2': 0.6, 'x1': 0.5, 'x3': 0.4}
```

Practical no.09

Aim: Create fuzzy relation by Cartesian product of any two fuzzy sets

Code:

```
import pandas as pd
def fuzzy_cartesian_product(A,B):
    R = {}
    for x,mu_x in A.items():
        for y,mu_y in B.items():
            R[(x,y)] = min(mu_x,mu_y)
    return R

if __name__== "__main__":
    A = {"x1": 0.2, "x2": 0.7}
    B = {"y1": 0.5, "y2": 0.4, "y3": 1.0}

    R = fuzzy_cartesian_product(A, B)
    print("Fuzzy Set A:", A)
    print("Fuzzy Set B:", B)
    print("\nFuzzy Relation (A x B) as pairs:")

    for pair, val in R.items():
        print(f"{pair}: {val}")
    print("\nFuzzy Relation (Matrix form):")
    matrix = pd.DataFrame(
        [[R[(x, y)] for y in B] for x in A],
    index=A.keys(),
    columns=B.keys()
    )
    print(matrix)
```

Output:

```
= RESTART: C:/Users/CS/AppData/Local/Programs/Python/Python314/practical 9 sc.py
Fuzzy Set A: {'x1': 0.2, 'x2': 0.7}
Fuzzy Set B: {'y1': 0.5, 'y2': 0.4, 'y3': 1.0}

Fuzzy Relation (A x B) as pairs:
('x1', 'y1'): 0.2
('x1', 'y2'): 0.2
('x1', 'y3'): 0.2
('x2', 'y1'): 0.5
('x2', 'y2'): 0.4
('x2', 'y3'): 0.7

Fuzzy Relation (Matrix form):
     y1   y2   y3
x1  0.2  0.2  0.2
x2  0.5  0.4  0.7
```

Practical No.10

Aim: Perform max-min composition on any two fuzzy relations.

Code:

```
import numpy as np
import pandas as pd

def max_min_composition(R1, R2):
    R1 = np.array(R1)
    R2 = np.array(R2)

    if R1.shape[1] != R2.shape[0]:
        raise ValueError(
            "Incompatible relations: columns of R1 must be equal to rows of R2"
        )

    result = np.zeros((R1.shape[0], R2.shape[1]))

    for i in range(R1.shape[0]):
        for j in range(R2.shape[1]):
            for k in range(R1.shape[1]):
                result[i, j] = max(
                    result[i, j],
                    min(R1[i, k], R2[k, j])
                )

    return result


if __name__ == "__main__":
    R1 = [
        [0.2, 0.7, 1.0],
        [0.5, 0.4, 0.6]
    ]

    R2 = [
        [0.6, 0.2],
        [0.8, 0.5],
        [0.3, 0.9]
    ]

    R = max_min_composition(R1, R2)

    print("Relation R1 (X-Y):")
    print(pd.DataFrame(R1))

    print("\nRelation R2 (Y-Z):")
    print(pd.DataFrame(R2))
```

```
print("\nMax-Min Composition (R1 o R2) (X-Z):")
print(pd.DataFrame(R))
```

Output:

```
= RESTART: C:/Users/91908/AppData/Local/Programs/Py
Relation R1 (X-Y):
     0    1    2
0  0.2  0.7  1.0
1  0.5  0.4  0.6

Relation R2 (Y-Z):
     0    1
0  0.6  0.2
1  0.8  0.5
2  0.3  0.9

Max-Min Composition (R1 o R2) (X-Z):
     0    1
0  0.7  0.9
1  0.5  0.6
>
```